DATABASE MANAGEMENT SYSTEM (CT 652)

Tribhuvan University | Institute of Engineering | BCT | Year III / Part II

CHAPTER 5: Transaction Management and Concurrency Control

Syllabus Hours: 7 hrs | Marks Weight: 14 marks

Complete Exam-Ready Answers | All Past Questions Covered (2070–2082)

CHAPTER 5 SYLLABUS TOPICS

- **5.1** Transaction Concept and Transaction State
- **5.2** Implementation of Atomicity and Durability
- **5.3** Concurrent Executions and ACID Properties
- **5.4** Serializability (Conflict and View Serializability)
- **5.5** Recoverability — Recoverable, Cascadeless, Strict Schedules
- **5.6** Lock-Based Protocols (Shared/Exclusive, 2PL, Strict 2PL)
- **5.7** Timestamp-Based Protocols
- **5.8** Deadlock Handling — Prevention, Detection, Recovery
- **5.9** Recovery System — Log-Based Recovery, Checkpoints

SECTION 1: Transaction Concept, ACID Properties & Transaction States

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q1. What is a transaction in DBMS? Explain the ACID properties of a transaction with examples. Why is each property important?

[2+4+2]
2081 Chaitra 2079 Chaitra 2075 Bhadra

Answer: Transaction Concept and ACID Properties

Transaction: A transaction is a logical unit of work in a database — a sequence of one or more database operations (reads and writes) that must be executed as a single, indivisible unit. A transaction either completes entirely (commits) or has no effect at all (aborts/rolls back). Example: transferring Rs. 5000 from Account A to Account B involves two operations (debit A, credit B) that must both succeed or both fail.

ACID Properties of Transactions:

Property	Full Name	Definition	Example (Bank Transfer: A→B, Rs.5000)
A	Atomicity	The transaction is treated as a single unit — either ALL operations complete successfully, or NONE of them take effect. No partial execution.	Either BOTH debit(A,5000) AND credit(B,5000) succeed, or NEITHER happens. The database is

			never left in a state where A is debited but B is not credited.
C	Consistency	A transaction brings the database from one VALID state to another valid state. All integrity constraints, rules, and invariants must be satisfied before and after the transaction.	Total money in the system (A+B) must be the same before and after the transfer. No money is created or destroyed by the transaction.
I	Isolation	Concurrent transactions execute as if they were running SERIALLY — each transaction is unaware of other simultaneously running transactions. Intermediate states are not visible to other transactions.	While the transfer is in progress, no other transaction should see A debited but B not yet credited. Concurrent transactions see a consistent view of data.
D	Durability	Once a transaction COMMITS, its changes are permanent — they persist even in the event of system failures (power outage, crash, hardware failure).	After the transfer commits, the new balances of A and B remain even if the system crashes immediately after the commit. Changes are written to persistent storage (disk).

Most Repeated: "Define transaction. Explain ACID properties with examples." appears in virtually every exam. Always give a concrete banking example for each property — it's the most expected illustration.

Q2. Explain the various states of a transaction with a state transition diagram. What happens at each state?

[2+6]

2081 Ashwin 2079
Chaitra 2076 Baisakh

Answer: Transaction States

A transaction goes through a sequence of states during its lifetime. The state transition model captures what happens from when a transaction begins until it is fully completed or aborted.

State	When Entered	Description	Next States
Active	Transaction begins execution.	The transaction is currently executing — performing read and write operations on the database. The INITIAL state of every transaction.	Partially Committed, Failed
Partially Committed	Final operation (last statement) of the transaction has been executed.	The transaction has executed all its statements, but changes may still be in the buffer (not yet written to disk). Consistency checks are performed.	Committed, Failed

Committed	All consistency checks pass; transaction writes to disk.	The transaction has successfully completed. All changes are permanently written to the database. Changes become visible to other transactions.	Terminated
Failed	An error occurs during Active or Partially Committed state.	The transaction cannot proceed normally — hardware failure, software error, integrity violation, or explicit ROLLBACK by the user.	Aborted
Aborted	Recovery manager rolls back all changes.	All changes made by the failed transaction are UNDONE (rolled back). The database is restored to the state it was in before the transaction began.	Active (restart), Terminated (killed)
Terminated	After Committed or Aborted.	The transaction has ended completely — either successfully committed or aborted. It is no longer active.	(End)

State Transition Diagram:

Transaction State Transitions

```
BEGIN TRANSACTION
      ↓
[ACTIVE] ──────── error / ROLLBACK ──────────→ [FAILED]
      ↓ (last operation executed)                    ↓ (UNDO all changes)
[PARTIALLY COMMITTED]                          [ABORTED]
      ↓ (writes to disk succeed)                     ↓ (restart OR kill)
[COMMITTED]                                    [TERMINATED]
      ↓
[TERMINATED]
```

Result: Active and Partially Committed can both transition to Failed on any error.

SECTION 2: Concurrent Execution, Schedules & Serializability

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q3. Why is concurrent execution of transactions needed? What problems can arise when transactions execute concurrently? Explain with examples.

[2+6]
2080 Ashwin 2078 Chaitra 2074 Bhadra

Answer: Concurrent Execution and Problems

Concurrent execution means multiple transactions are running in an interleaved manner — the CPU switches between transactions, executing a few operations from each. This is essential in modern DBMS for efficiency.

Why Concurrent Execution is Needed:

- Improved CPU Utilization: While one transaction waits for disk I/O, the CPU can process another transaction.
- Higher Throughput: More transactions complete per unit of time.
- Reduced Waiting Time: Short transactions do not have to wait for long transactions to complete.
- Better Resource Usage: Memory, CPU, and I/O are all kept busy simultaneously.

Problems Caused by Concurrent Execution (Concurrency Problems):

Problem	Also Called	Description	Example
Lost Update Problem	Write-Write Conflict	Two transactions read the same value and both update it. The second write overwrites the first, so the first transaction's update is LOST.	T1 reads X=100, T2 reads X=100. T1 writes X=150, T2 writes X=80. T1's update is lost — net effect is wrong.
Dirty Read Problem	Read Uncommitted / WR Conflict	A transaction reads data written by another transaction that has NOT yet committed. If the writing transaction aborts, the reader has read "dirty" (invalid) data.	T1 updates X=200. T2 reads X=200 (dirty). T1 aborts → X reverts to 100. But T2 used the incorrect value 200 in its computation.
Non-Repeatable Read	Read-Write / RW Conflict	A transaction reads the same data item twice but gets different values because another committed transaction updated it in between the two reads.	T1 reads X=100. T2 updates X=200 and commits. T1 reads X again and gets 200. T1 sees two different values for X in the same transaction.
Phantom Read	Phantom Problem	A transaction re-executes a query and finds different rows (new rows inserted or existing rows deleted by another committed transaction) that were not there before.	T1 queries all employees with salary > 50K (finds 10). T2 inserts a new employee with 60K salary and commits. T1 re-queries and finds 11. A "phantom" row appeared.
Incorrect Summary	Aggregate Inconsistency	One transaction computes an aggregate (SUM, COUNT) while another modifies the values being aggregated. The result may be incorrect.	T1 is summing all account balances. T2 is transferring money between accounts simultaneously. T1 may read the deducted value from one account but not yet the added value in another.

Q4. What is a schedule? Define serial and concurrent (non-serial) schedules. What is a serializable schedule? Why is serializability important?

[1+2+2+3]
2081 Chaitra 2079
Chaitra 2075 Bhadra

Answer: Schedules and Serializability

Schedule: A schedule is a sequence of the instructions (read, write, commit, abort) of all concurrent transactions in the order in which they are executed. A schedule represents the actual order of execution of operations from multiple transactions.

Type	Definition	Properties	Example
Serial Schedule	All operations of one transaction complete before any operation of the next transaction begins. Transactions run one after another — NO interleaving.	Always consistent. Never causes concurrency problems. But slow — no concurrency benefit.	T1 runs completely, then T2 runs completely: T1(R(A),W(A),Commit), T2(R(A),W(A),Commit)
Concurrent (Non-Serial) Schedule	Operations from multiple transactions are INTERLEAVED — they run simultaneously with operations from different transactions mixed together.	May or may not be consistent. Faster — allows concurrency. May cause dirty reads, lost updates, etc.	R1(A), R2(A), W1(A), W2(A), C1, C2 — operations from T1 and T2 interleaved.
Serializable Schedule	A concurrent schedule whose effect on the database is identical to SOME serial schedule of the same transactions.	Correct AND concurrent — achieves consistency of a serial schedule with the efficiency of concurrent execution.	A concurrent schedule equivalent to serial T1→T2 or T2→T1.

Serializability is the GOLD STANDARD for correctness in concurrent transaction execution — it guarantees that concurrent execution produces a correct (consistent) result while still allowing the efficiency benefits of concurrency.

Q5. What is Conflict Serializability? Define conflicting operations. Explain the Precedence Graph (Serialization Graph) method to test for conflict serializability, with examples.

[2+4+4]
2082 Kartik 2079
Chaitra 2077 Chaitra

Answer: Conflict Serializability and Precedence Graph

Conflicting Operations: Two operations from different transactions CONFLICT if and only if: (1) They belong to DIFFERENT transactions, AND (2) They access the SAME data item, AND (3) At least ONE of them is a WRITE operation.

Operation Pair	Same Data Item?	At Least One Write?	Conflict?
R_i(X) and R_j(X)	Yes	No (both reads)	NO — reads never conflict
R_i(X) and W_j(X)	Yes	Yes (W_j writes)	YES — Read-Write conflict
W_i(X) and R_j(X)	Yes	Yes (W_i writes)	YES — Write-Read conflict
W_i(X) and W_j(X)	Yes	Yes (both write)	YES — Write-Write conflict
R_i(X) and W_j(Y)	No (X≠Y)	—	NO — different data items

Conflict Equivalent Schedules: Two schedules S1 and S2 are conflict equivalent if S2 can be obtained from S1 by swapping non-conflicting operations of different transactions. S1 is conflict serializable if it is conflict equivalent to some serial schedule.

Precedence Graph (Serialization Graph) Test:

Algorithm to test if a schedule S is conflict serializable:

- Step 1: Create one NODE for each transaction Ti in S.
- Step 2: Draw a directed edge Ti → Tj (means Ti must come before Tj) if:
- (a) Ti writes X before Tj reads X [W-R conflict: Ti → Tj]
- (b) Ti reads X before Tj writes X [R-W conflict: Ti → Tj]
- (c) Ti writes X before Tj writes X [W-W conflict: Ti → Tj]
- Step 3: If the precedence graph has NO CYCLE → Schedule is CONFLICT SERIALIZABLE. ✓
- Step 4: If there IS A CYCLE → NOT conflict serializable. ✗

Example 1 — Schedule S1 (Conflict Serializable):

Schedule S1: T1 and T2
T1: R(A) W(A) C1
T2: R(B) W(B) R(A) W(A) C2

- T1 W(A) before T2 R(A) → Edge: T1 → T2 (W-R conflict)
- T1 W(A) before T2 W(A) → Edge: T1 → T2 (W-W conflict, same direction)
- Precedence Graph: T1 → T2 (no cycle)
- CONFLICT SERIALIZABLE ✓ Equivalent to serial schedule T1, T2.

Example 2 — Schedule S2 (NOT Conflict Serializable):

Schedule S2: T1 and T2
T1: R(A) W(A) C1

```
T2:        W(A)        R(A)   W(A)   C2
```

- T2 W(A) before T1 W(A) → Edge: T2 → T1 (W-W conflict)
- T1 W(A) before T2 R(A) → Edge: T1 → T2 (W-R conflict)
- Precedence Graph: T1 → T2 AND T2 → T1 → CYCLE DETECTED!
- NOT CONFLICT SERIALIZABLE ✗

Example 3 — Three Transactions (Common Exam Pattern):

Schedule S3: T1, T2, T3
`T1: R(X)  W(X)  C1`
`T2:  R(Y)  W(Y)  C2`
`T3:  R(X)  R(Y)  W(Y)  C3`

- T1 R(X) before T3 R(X): No conflict (both reads).
- T1 W(X) — T3 R(X): Is T1 W(X) before T3 R(X)? Yes → T1 → T3 (W-R)
- T2 R(Y) before T3 W(Y) → T2 → T3 (R-W)
- T2 W(Y) before T3 W(Y) → T2 → T3 (W-W, same direction)
- Precedence Graph: T1→T3, T2→T3 (no cycle)
- CONFLICT SERIALIZABLE ✓ Equivalent serial order: T1,T2,T3 or T2,T1,T3.

Q6. What is View Serializability? How does it differ from Conflict Serializability? Explain the conditions for view equivalence.

[2+4+2]
2079 Ashwin Back
2078 Chaitra 2073 Bhadra

Answer: View Serializability

View Equivalent Schedules: Two schedules S1 and S2 are view equivalent if for every data item X: (1) INITIAL READ: If Ti reads the initial value of X in S1, Ti also reads the initial value of X in S2. (2) UPDATED READ: If Ti reads the value of X written by Tj in S1, then Ti reads the value written by Tj in S2 as well. (3) FINAL WRITE: If Ti performs the final (last) write of X in S1, then Ti also performs the final write of X in S2.

View Serializable Schedule: A schedule S is VIEW SERIALIZABLE if it is view equivalent to some serial schedule.

Feature	Conflict Serializability	View Serializability
Definition	Schedule equivalent to a serial schedule by swapping non-conflicting ops.	Schedule with same initial reads, updated reads, and final writes as a serial schedule.

Testing Method	Precedence graph — check for cycles. Simple and efficient.	Check three conditions: initial read, updated read, final write. More complex.
Strictness	Stricter (subset of view serializable).	Less strict — all conflict serializable schedules are also view serializable, but NOT vice versa.
Blind Writes	Cannot handle (blind write causes a cycle).	Can handle blind writes (a write not preceded by a read of same item).
Practical Use	Widely used — basis for lock protocols.	Theoretical importance; rarely implemented directly due to complexity.
Relationship	Every conflict serializable schedule is also view serializable.	Some view serializable schedules are NOT conflict serializable (contain blind writes).

Exam Tip: "Blind write" is a write operation that is NOT preceded by a read of the same item by the same transaction. Schedules with blind writes can be view serializable but NOT conflict serializable. This distinction is frequently tested.

SECTION 3: Recoverability — Recoverable, Cascadeless & Strict Schedules

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2079 Chaitra, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2073 Bhadra, 2072 Ashwin

Q7. What is recoverability of a schedule? Define and compare: Recoverable Schedule, Cascadeless Schedule, and Strict Schedule. Give an example of each.

[2+6]
2081 Ashwin 2079 Chaitra 2075 Bhadra

Answer: Recoverability in Schedules

A schedule is said to be RECOVERABLE if transactions can be rolled back properly after a failure — ensuring the database can always be restored to a consistent state. Recoverability deals with the READ-WRITE dependencies between transactions (specifically, what happens when a transaction reads data written by another transaction that later aborts).

The Problem — Unrecoverable Schedule:

Unrecoverable Schedule Example				
T1:	W(A)			Abort
T2:		R(A)	W(B)	Commit

Result: T2 reads dirty data from T1 (W-R dependency). T2 commits BEFORE T1 aborts. Cannot undo T2 — UNRECOVERABLE! ✗

Schedule Type	Definition	Requirement	Cascade Possible?
Recoverable Schedule	If transaction Tj reads data written by Ti, then Ti must COMMIT before Tj commits. No transaction commits based on dirty data from an uncommitted transaction.	Ti commits BEFORE Tj commits (for all W_i(X) → R_j(X) dependencies).	Yes — cascading rollbacks possible
Cascadeless (Avoids Cascading Aborts) Schedule	A transaction Tj is only allowed to read data written by Ti AFTER Ti has committed. Prevents dirty reads entirely.	Ti commits BEFORE Tj reads Ti's written value.	No — no cascading rollbacks possible
Strict Schedule	No transaction may read OR write a data item written by another transaction until the first transaction has committed or aborted. Most restrictive.	Ti commits or aborts BEFORE Tj reads OR writes any item written by Ti.	No — also simplifies recovery (just restore old values)

Hierarchy of Recoverability (from least to most restrictive):

Schedule Hierarchy
`Serializable ⊆ Strict ⊆ Cascadeless ⊆ Recoverable ⊆ All Schedules`
`Every strict schedule is also cascadeless.`
`Every cascadeless schedule is also recoverable.`
`But NOT vice versa in each case.`

Examples:

Example 1 — Recoverable (but allows cascading rollback)
`T1: W(A)  Commit`
`T2:  R(A)  Commit`
Result: T1 commits BEFORE T2 commits. Recoverable ✓ — but if T1 aborted before commit, T2 must also be rolled back (cascade). T2 cannot commit until T1 commits.

Example 2 — Cascadeless (no dirty reads)
`T1: W(A)  Commit`
`T2:  R(A)  Commit`
Result: T2 reads A only AFTER T1 commits. No dirty read possible. Cascadeless ✓. Also Recoverable ✓.

Example 3 — Strict Schedule
`T1: W(A)  Commit`
`T2:  R(A)  W(A)  Commit`
Result: T2 neither reads NOR writes A until AFTER T1 commits. Strict ✓. Also Cascadeless ✓ and Recoverable ✓.

SECTION 4: Lock-Based Protocols

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q8. What are locks in DBMS? Explain Shared Lock and Exclusive Lock with the lock compatibility matrix. What is a lock protocol? **[2+4+2]** *2081 Chaitra 2079 Chaitra 2075 Bhadra*

Answer: Locks and Lock Compatibility

Lock: A lock is a mechanism to control concurrent access to a data item. When a transaction needs to read or write a data item, it must first acquire the appropriate lock. Other transactions requesting conflicting locks must wait — ensuring isolation.

Lock Type	Symbol	Request When	Granted If	Allows Others To
Shared Lock (S-Lock)	S	Transaction wants to READ a data item.	No transaction holds an Exclusive lock on the item.	Read the same item (multiple shared locks coexist). But NOT write.
Exclusive Lock (X-Lock)	X	Transaction wants to WRITE (or read-then-write) a data item.	NO other transaction holds ANY lock (S or X) on the item.	Neither read nor write until X-lock is released. Complete exclusive access.

Lock Compatibility Matrix:

Can a new lock request be GRANTED if the data item already has an existing lock?

Existing Lock \ Requested	S (Shared)	X (Exclusive)
S (Shared already held)	**Yes**	**No**
X (Exclusive already held)	**No**	**No**
None (no lock)	**Yes**	**Yes**

Reading: "S-Yes" means if a Shared lock is already held, a new Shared request can be GRANTED. "S-No" for Exclusive means if Shared is already held, an Exclusive request must WAIT.

- Two transactions CAN both hold S-locks on the same item simultaneously (compatible).
- An X-lock is INCOMPATIBLE with any other lock (S or X) — only one transaction can hold X at a time.

Lock Protocol: A lock protocol (locking protocol) is a set of rules specifying when a transaction may lock and unlock data items. The protocol ensures that the resulting schedules are serializable and/or have other desirable properties.

Q9. What is Two-Phase Locking (2PL)? Explain the two phases of 2PL. Prove that 2PL guarantees conflict serializability. Also explain Strict 2PL and Rigorous 2PL.

[2+4+2]
2080 Ashwin 2078 Chaitra 2073 Bhadra

Answer: Two-Phase Locking Protocol (2PL)

Two-Phase Locking (2PL): In 2PL, a transaction is divided into exactly two phases: a GROWING phase (can only acquire locks, cannot release any) and a SHRINKING phase (can only release locks, cannot acquire any new ones). A transaction cannot acquire a new lock after it has released any lock.

Phase	Name	Allowed Operations	Description
Phase 1	Growing Phase (Lock Acquisition)	ONLY acquire locks (S or X). NO lock release.	Transaction requests and acquires all locks it needs. The number of held locks grows from 0 to maximum.
Phase 2	Shrinking Phase (Lock Release)	ONLY release locks. NO new lock acquisition.	Transaction releases locks one by one as it no longer needs them. The number of held locks decreases to 0.

Lock Point:

Lock Point: The point during transaction execution at which the transaction has acquired ALL the locks it needs (the transition from growing to shrinking phase). The lock point determines the serialization order.

Why 2PL Guarantees Conflict Serializability:

Proof (by contradiction): Suppose schedule S follows 2PL but is NOT conflict serializable. Then the precedence graph of S has a cycle: T1 → T2 → ... → Tk → T1. This means T1 must have its lock point before T2, T2 before T3, ..., Tk before T1. But this creates a contradiction (T1 before T1). Therefore, 2PL cannot produce a cycle → 2PL always produces conflict serializable schedules. ✓

Variants of 2PL:

Variant	Description	Lock Release Rule	Prevents Deadlock?	Cascading Rollback?
Basic 2PL	Standard 2PL as described above.	Locks can be released during transaction (shrinking phase).	No	Yes (dirty reads possible)
Strict 2PL	All EXCLUSIVE locks held until transaction commits or aborts.	X-locks released only at commit/abort.	No	No (no dirty reads)
Rigorous 2PL	ALL locks (both S and X) held until commit or abort.	All locks released only at commit/abort. Simplest to implement.	No	No — most restrictive
Conservative 2PL	All locks acquired BEFORE transaction begins (pre-declaration).	Locks released normally.	Yes	No

Most Repeated: "Explain 2PL with its two phases. Does 2PL always give conflict serializable schedules?" is asked almost every year. The key answer: YES — 2PL guarantees conflict serializability. But basic 2PL does NOT prevent deadlocks.

Example — 2PL Schedule (Strict 2PL):

Strict 2PL: T1 transfers Rs.500 from A to B

```
T1: Lock-X(A)  R(A)  W(A)  Lock-X(B)  R(B)  W(B)  Commit  Unlock(A)
Unlock(B)

    [--- Growing Phase ---]            [Still Growing]       [Shrinking
starts at commit]
```

Result: All exclusive locks held until commit. Strict 2PL ensures no dirty reads and supports clean recovery.

SECTION 5: Deadlock — Detection, Prevention & Recovery

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin

Q10. What is a deadlock? How does it occur in a database system? Explain the different methods of deadlock handling: Prevention, Detection, and Recovery.	**[2+2+6]** *2081 Chaitra 2079 Chaitra 2076 Baisakh*

Answer: Deadlock Handling

Deadlock: A deadlock is a situation where a set of transactions are each waiting for a lock held by another transaction in the set — creating a circular waiting chain. None of the transactions can proceed, and they will wait forever unless external intervention occurs.

Classic Deadlock Example:

Deadlock Scenario: T1 and T2
`T1: Lock-X(A)  ...  Wait for Lock-X(B) ← blocked by T2`
`T2: Lock-X(B)  ...  Wait for Lock-X(A) ← blocked by T1`
`T1 holds A, wants B.`
`T2 holds B, wants A.`
`Neither can proceed → DEADLOCK!`
Result: Circular wait: T1 → T2 → T1. Both transactions wait forever.

1. Deadlock Prevention:

Prevent deadlocks by ensuring that at least one of the four necessary conditions for deadlock (mutual exclusion, hold and wait, no preemption, circular wait) can never hold.

Prevention Method	Description	Drawback
Pre-declaration (Conservative 2PL)	Every transaction locks ALL data items it will need BEFORE it begins execution. If any lock is unavailable, it waits without holding any lock.	Difficult to know all needed items in advance. Low concurrency.
Wait-Die Scheme (Non-preemptive)	If Ti requests a lock held by Tj: if Ti is OLDER than Tj → Ti waits (old waits for young). If Ti is YOUNGER → Ti is killed (dies) and restarts.	Younger transactions may be repeatedly aborted.

Wound-Wait Scheme (Preemptive)	If Ti requests a lock held by Tj: if Ti is OLDER → Tj is killed (wounded) and restarted. If Ti is YOUNGER → Ti waits.	Older transactions may preemptively kill younger ones.
Timeout Scheme	Each transaction is given a maximum time limit (timeout). If it does not acquire a lock within the timeout, it is aborted and restarted.	Risk of unnecessary aborts. Starvation if timeout is too short.

Wait-Die vs Wound-Wait Summary:

	Wait-Die	Wound-Wait
Older Ti requests lock held by Younger Tj	Ti WAITS (old waits for young)	Tj is PREEMPTED/killed (old wounds young)
Younger Ti requests lock held by Older Tj	Ti DIES (young dies rather than waiting)	Ti WAITS (young waits for old)
Type	Non-preemptive (running transaction not aborted)	Preemptive (running transaction may be aborted)
Memory trick	Wait = Old waits; Die = Young dies	Wound = Old wounds young; Wait = Young waits

2. Deadlock Detection:

Allow deadlocks to occur but detect them using a Wait-For Graph (WFG) and then recover.

Wait-For Graph (WFG): A directed graph where nodes are transactions. A directed edge Ti → Tj exists if Ti is waiting for a lock held by Tj. If the WFG has a CYCLE, a deadlock exists. The DBMS periodically runs a cycle detection algorithm on the WFG.

Wait-For Graph Example
`Transactions: T1, T2, T3, T4`
`T1 waits for T2 → Edge: T1 → T2`
`T2 waits for T3 → Edge: T2 → T3`
`T3 waits for T1 → Edge: T3 → T1`
`T4 waits for T2 → Edge: T4 → T2`
`WFG: T1 → T2 → T3 → T1  (CYCLE! T4 → T2 — T4 not in cycle)`

Result: Deadlock detected among T1, T2, T3. T4 is just waiting, not in deadlock.

3. Deadlock Recovery:

Once a deadlock is detected, the system must break it by selecting a VICTIM — a transaction in the deadlock cycle to abort and rollback.

Recovery Step	Description
1. Select a Victim	Choose a transaction to abort. Selection criteria: lowest cost to rollback (fewest updates made), oldest or youngest transaction, transaction that has been restarted most times.
2. Rollback the Victim	Abort the selected victim transaction and undo all its changes using the log. Release all its locks. Other transactions in the deadlock can now proceed.
3. Restart or Kill	Either restart the victim transaction (if it was aborted not because of user intent) or kill it permanently (if it caused the deadlock multiple times → starvation prevention).
4. Total vs Partial Rollback	Total rollback: undo the entire transaction. Partial rollback: rollback only enough to break the deadlock. Partial rollback requires the system to know safe rollback points (savepoints).
5. Starvation Prevention	Ensure the same transaction is not always chosen as victim. Include rollback count in cost calculation — transaction that has been rolled back many times should not be chosen again.

SECTION 6: Timestamp-Based Protocols

Exam Ref: 2082 Kartik, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin

Q11. What is a Timestamp? Explain the Timestamp Ordering Protocol. How does it ensure serializability? What are the rules for READ and WRITE operations?	**[2+6]** *2079 Chaitra 2078 Chaitra 2074 Bhadra*

Answer: Timestamp-Based Protocols

Timestamp: A timestamp is a unique identifier assigned to each transaction when it starts — representing the time at which the transaction began. Timestamps are used to establish a serialization order: if TS(Ti) < TS(Tj), then Ti is treated as if it started before Tj and should be serialized before Tj.

Timestamps Maintained per Data Item X:

Timestamp	Full Form	Meaning
W-timestamp(X)	Write timestamp of X	The timestamp of the YOUNGEST (most recent) transaction that SUCCESSFULLY WROTE data item X.
R-timestamp(X)	Read timestamp of X	The timestamp of the YOUNGEST (most recent) transaction that SUCCESSFULLY READ data item X.

Timestamp Ordering (TO) Protocol Rules:

For transaction Ti with timestamp TS(Ti):

READ Operation — Ti wants to read data item X:

Condition	Action	Reason
TS(Ti) < W-timestamp(X)	REJECT read. Rollback Ti and restart with a new (larger) timestamp.	Ti is trying to read a value that a YOUNGER transaction has already overwritten. Too late to read the old value.
TS(Ti) ≥ W-timestamp(X)	ALLOW read. Set R-timestamp(X) = max(R-timestamp(X), TS(Ti)).	Ti is reading the most recent value written before Ti started. This is correct.

WRITE Operation — Ti wants to write data item X:

Condition	Action	Reason
TS(Ti) < R-timestamp(X)	REJECT write. Rollback Ti and restart.	A younger transaction has already READ X. Ti's write would invalidate that read — too late.
TS(Ti) < W-timestamp(X)	REJECT write. Rollback Ti and restart. (Thomas Write Rule: can IGNORE this write instead)	A younger transaction has already WRITTEN X. Ti's write is outdated — the value Ti would write is obsolete.
Otherwise (TS(Ti) ≥ both)	ALLOW write. Set W-timestamp(X) = TS(Ti).	Ti's write is the most recent and valid.

Thomas Write Rule (Optimization):

Thomas Write Rule: If TS(Ti) < W-timestamp(X) during a WRITE operation, Ti's write is OBSOLETE (a more recent transaction has already written a newer value). Instead of aborting Ti,

simply IGNORE Ti's write (skip it). This allows more schedules to proceed without aborting — produces view serializable (not always conflict serializable) schedules.

Properties of Timestamp Ordering Protocol:

Property	Value
Deadlock-Free	YES — no transaction ever waits for another. Either proceeds or is aborted and restarted.
Starvation Possible	YES — a transaction may be repeatedly aborted if it always gets a timestamp smaller than competing transactions.
Serializability	YES — TO ensures conflict serializability (equivalent to serial order by timestamp values).
Livelock	Possible — if a transaction is repeatedly restarted and always conflicts. Use priority queues to prevent.

Q12. Compare Lock-Based Protocol (2PL) with Timestamp-Based Protocol. When would you prefer one over the other?

[4+4]

2079 Ashwin Back
2078 Baishakh 2072 Ashwin

Answer: 2PL vs Timestamp Protocol — Comparison

Feature	Two-Phase Locking (2PL)	Timestamp Ordering (TO)
Mechanism	Uses locks (S-lock and X-lock) to control access. Waits when conflict.	Uses timestamps assigned at transaction start. No locks, no waiting.
Deadlock	YES — deadlocks possible. Detection/prevention needed.	NO deadlocks — transactions never wait (abort instead).
Starvation	Less common (fairness via lock queues).	Possible — old transactions may be repeatedly aborted.
Serializability	Conflict serializable. Serial order = order of lock points.	Conflict serializable. Serial order = order of timestamps.
Overhead	Lock management overhead (granting, releasing, checking compatibility).	Timestamp comparison overhead (checking W-ts and R-ts per operation).
Cascading Rollback	Possible with basic 2PL (Strict 2PL prevents it).	Possible — a rolled-back transaction may have had its writes read by others.

Implementation	Widely implemented in commercial RDBMS (Oracle, MySQL InnoDB, PostgreSQL).	Used in some distributed DBs and optimistic protocols; less common than 2PL.
Best For	Systems with many transactions accessing same data (high contention).	Systems where conflicts are rare (low contention); distributed databases.

SECTION 7: Recovery System — Log-Based Recovery and Checkpoints

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin

Q13. What is the Recovery System in DBMS? Explain Log-Based Recovery with the structure of a log record. What are UNDO and REDO operations? What is the difference between Deferred Update and Immediate Update?

[2+4+2]
2081 Chaitra 2079 Chaitra 2075 Bhadra

Answer: Recovery System and Log-Based Recovery

The Recovery System ensures that the database returns to a consistent state after a failure (system crash, transaction abort, media failure). It uses a LOG (also called journal or audit trail) to record all database modifications so that any changes can be undone (UNDO) or redone (REDO) as needed.

Structure of a Log Record:

Log Record Type	Fields	Meaning
Start Record	<Ti, start>	Transaction Ti has started execution.
Write Record	<Ti, X, old_value, new_value>	Transaction Ti has updated data item X from old_value to new_value.
Read Record (optional)	<Ti, X>	Transaction Ti has read data item X (not always logged).
Commit Record	<Ti, commit>	Transaction Ti has successfully completed and committed all changes.
Abort Record	<Ti, abort>	Transaction Ti has been aborted (all changes must be undone).
Checkpoint Record	<checkpoint, [Ti, Tj, ...]>	System has written all committed changes to disk. Lists active transactions at checkpoint time.

UNDO and REDO Operations:

Operation	When Used	Action Performed
UNDO (Ti)	Transaction Ti has ABORTED, or has not yet committed at time of failure.	Use old_value from log records to RESTORE all data items modified by Ti to their previous values. Scan log BACKWARDS from the end.
REDO (Ti)	Transaction Ti has COMMITTED but changes may not yet be on disk (lost due to crash).	Use new_value from log records to RE-APPLY all updates of Ti. Scan log FORWARDS from the beginning.

Recovery Procedure After a System Crash:

- Step 1: Read the log from most recent checkpoint.
- Step 2: Classify transactions: REDO list = those with <Ti, commit> in log. UNDO list = those without a commit record.
- Step 3: REDO all committed transactions (scan log forward, apply new_values).
- Step 4: UNDO all uncommitted transactions (scan log backward, apply old_values).

Deferred Update vs Immediate Update:

Feature	Deferred Update (NO-UNDO/REDO)	Immediate Update (UNDO/REDO)
When changes written	Changes are NOT written to disk until transaction commits. Written to log first (write-ahead logging).	Changes are written to disk IMMEDIATELY during execution (may be before commit).
UNDO needed?	NO — uncommitted changes never reach disk. On abort, just discard log.	YES — if crash before commit, must undo partial changes already on disk.
REDO needed?	YES — committed changes may be in log but not yet on disk.	YES — committed changes may have been on disk but lost in crash (buffer not flushed).
Recovery simplicity	Simpler — no undo needed.	More complex — both undo and redo needed.
Disadvantage	Transactions need extra memory to hold all deferred changes.	More complex recovery; requires old_value in log for undo.

Q14. What is a Checkpoint in database recovery? Why are checkpoints needed? Explain how checkpoints improve recovery performance with an example showing which transactions are UNDO'd and REDO'd after a crash.	**[2+2+4]** *2080 Ashwin 2078 Baishakh 2073 Bhadra*

Answer: Checkpoints in Recovery

Checkpoint: A checkpoint is a mechanism where the DBMS periodically forces all modified data from the buffer (memory) to disk and writes a special <checkpoint> record to the log. The checkpoint record lists all active (uncommitted) transactions at the time of the checkpoint. After a checkpoint, log records before it do not need to be examined during recovery.

Why Checkpoints Are Needed:

- Without checkpoints, recovery after a crash must scan the ENTIRE log from the beginning — very slow for large logs.
- With checkpoints, recovery only needs to examine the log from the most recent checkpoint — dramatically reduces recovery time.
- Checkpoints also allow old portions of the log to be truncated (deleted from disk), saving storage.

Recovery Using Checkpoints — Example:

Transaction Timeline and Checkpoint
`Log entries (time goes left to right):`
`<T1, start>`
`<T1, A, 100, 200>`
`<T2, start>`
`..... << CHECKPOINT >> (Active: T1, T2)`
`<T1, commit>  ← T1 committed AFTER checkpoint`
`<T3, start>`
`<T2, B, 50, 75>`
`<T4, start>`
`<T4, C, 10, 20>`
`<T3, commit>`
`..... << SYSTEM CRASH >>`

Recovery Analysis After Crash:

Transaction	Status at Crash	Action Required	Reason
T1	Committed (after checkpoint)	REDO T1	Has commit record in log after checkpoint. Changes may not be on disk.
T2	NOT committed (no commit record)	UNDO T2	Started before checkpoint. Was active at

			checkpoint. No commit record found.
T3	Committed (after checkpoint)	REDO T3	Has commit record in log after checkpoint.
T4	NOT committed (no commit record)	UNDO T4	Started after checkpoint. No commit record found.

Transactions that committed BEFORE the checkpoint (before our example) are already guaranteed to be on disk — they can be ignored entirely during recovery. This is why checkpoints speed up recovery.

Fuzzy Checkpoints:

Fuzzy Checkpoint: A variant of checkpointing where the system does not stop all transaction processing during the checkpoint. Instead, it writes a begin_checkpoint record, continues processing, then writes end_checkpoint. Recovery uses begin_checkpoint as the starting point. Reduces performance disruption from checkpointing.

SECTION 8: Additional Topics — Isolation Levels, Implementation

Exam Ref: 2082 Kartik, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2075 Bhadra, 2073 Bhadra

Q15. What are the four SQL Isolation Levels? Explain each level and which concurrency problems they prevent. Why would a programmer choose a lower isolation level?

[2+4+2]
2081 Ashwin 2079 Chaitra 2075 Bhadra

Answer: SQL Isolation Levels

SQL provides four standard isolation levels, allowing programmers to trade off consistency for performance. A lower isolation level allows more concurrency (and more performance) but risks certain concurrency anomalies.

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Implementation	Use Case
Read Uncommitted (Lowest)	Possible	Possible	Possible	No S-locks on reads. X-locks held until commit.	Reports where approximate data is acceptable. Rarely used.
Read Committed	Not Possible	Possible	Possible	S-locks acquired and released	Most common default (Oracle, PostgreSQL). Good balance.

				IMMEDIATELY after each read.	
Repeatable Read	Not Possible	Not Possible	Possible	S-locks held until commit. X-locks held until commit.	Financial data, when same read must return same value.
Serializable (Highest)	Not Possible	Not Possible	Not Possible	All S and X locks held until commit + range locks (index locking) to prevent phantoms.	Maximum correctness. Banking, medical records. Slowest.

Why choose a lower isolation level? Applications that only need approximate aggregates (e.g., reporting total website visitors) can use Read Uncommitted for maximum speed. Most OLTP applications use Read Committed. The lower the isolation level, the higher the concurrency and performance, but more care is needed in application logic.

Q16. What is the difference between a shadow paging recovery scheme and a log-based recovery scheme? Explain the shadow paging technique.

[2+6]

2079 Ashwin Back
2078 Baishakh 2073 Bhadra

Answer: Shadow Paging Recovery

Shadow Paging is an alternative to log-based recovery. Instead of logging changes, the system maintains TWO page tables: the CURRENT page table (pointing to current database pages) and a SHADOW page table (a copy pointing to the stable, pre-transaction version of pages).

Step	Action
1. Before Transaction Starts	Create a shadow page table = copy of current page table. Both tables point to the same physical pages initially.
2. During Transaction	When a page needs to be modified: allocate a NEW (fresh) disk page for the modified version. Write changes to the new page. Update CURRENT page table to point to the new page. SHADOW page table still points to the old page.
3. On Commit	Make the current page table permanent (write it to a stable/known location). The shadow page table is no longer needed. Discard the old (shadow) pages.
4. On Abort	Discard the current page table. Restore the shadow page table as the current page table. The database reverts to its pre-transaction state automatically.

Feature	Shadow Paging	Log-Based Recovery
How Atomicity Achieved	By atomically switching page tables at commit.	By logging and undo/redo operations.
Overhead Type	Page table copy overhead; disk space for old+new pages.	Log write overhead (sequential writes to log).
Recovery Speed	Fast — just switch page tables back.	Must process log records (undo/redo).
Fragmentation	Causes database fragmentation — new pages scattered.	No fragmentation (updates in place).
Concurrent Transactions	Difficult to support multiple concurrent transactions.	Easily supports many concurrent transactions.
Use Today	Rarely used in modern RDBMS (SQLite used a form of it).	Universally used in modern RDBMS (WAL - Write Ahead Log).

QUICK REVISION: Chapter 5 — Key Points Summary

Topic	Key Points	Expected Marks
Transaction & ACID	Transaction = logical unit of work. ACID: Atomicity (all or nothing), Consistency (valid state), Isolation (concurrent = serial), Durability (committed = permanent).	2-4
Transaction States	Active → Partially Committed → Committed → Terminated. Active / Partially Committed → Failed → Aborted → Terminated (or restart).	2-4
Concurrency Problems	Lost Update (W-W), Dirty Read (W-R uncommitted), Non-Repeatable Read (R-W), Phantom Read (insert/delete between reads).	2-4
Schedules	Serial: no interleaving, always correct. Concurrent: interleaved, faster. Serializable: concurrent but produces same result as some serial schedule.	2-4
Conflict Serializability	Two operations conflict if: diff transactions + same data item + at least one write. Precedence graph: no cycle = conflict serializable. Draw graph, check for cycles.	4-8
View Serializability	Same initial reads + updated reads + final writes as serial schedule. Less strict than conflict serializability. Handles blind writes.	2-4

Recoverability	Recoverable: Ti commits before Tj commits (WR dep). Cascadeless: Ti commits before Tj reads Ti's write. Strict: Ti commits before Tj reads OR writes Ti's data.	2-4
Locks & 2PL	S-lock: read, compatible with S. X-lock: write, incompatible with all. 2PL: growing phase (acquire only) + shrinking phase (release only). Lock point. Conflict serializable ✓. NO deadlock prevention ✗.	4-8
Strict/Rigorous 2PL	Strict 2PL: X-locks held until commit → no dirty reads. Rigorous 2PL: ALL locks held until commit.	2-4
Deadlock	Circular wait. Prevention: pre-declare, Wait-Die (old waits/young dies), Wound-Wait (old wounds/young waits), timeout. Detection: Wait-For Graph (cycle = deadlock). Recovery: choose victim and rollback.	4-8
Timestamp Protocol	TS(Ti) = start time. R-ts(X), W-ts(X). Read: reject if TS(Ti) < W-ts(X). Write: reject if TS(Ti) < R-ts(X) or W-ts(X). No deadlock ✓. Starvation possible ✗. Thomas Write Rule: ignore obsolete writes.	3-6
Recovery & Logs	Log record: <Ti, X, old, new>. UNDO: scan backward, apply old values (aborted txns). REDO: scan forward, apply new values (committed txns not yet on disk).	3-6
Checkpoints	Force buffer to disk + write checkpoint record. Recovery only reads log from last checkpoint. REDO: committed after checkpoint. UNDO: active at crash with no commit record.	2-4
Isolation Levels	Read Uncommitted → Read Committed → Repeatable Read → Serializable. Higher = more correct but less concurrent. Default in most RDBMS = Read Committed.	2-4

Best of Luck! | TU IOE BCT | CT 652 DBMS | Chapter 5: Transaction Management and Concurrency Control